

Conceito e Boas Práticas

- **Propósito:** As listas ajudam os usuários a encontrar itens específicos e realizar ações dinâmicas na interface.
- **Diretrizes:** Assegurar uma ordenação lógica (alfabética/numérica), manter textos objetivos, garantir consistência visual e escolher entre estilos como Padrão ou Segmentado.

Anatomia do ListItem

- **Container:** Estrutura que engloba o item (Obrigatório).
PDF
- **Headline (headlineContent):** Texto principal ou título do item (Obrigatório).
PDF
- **Overline (overlineContent):** Texto em destaque posicionado acima do título principal.
PDF
- **Supporting Text (supportingContent):** Descrição secundária ou detalhes.
PDF
- **Leading Content (leadingContent):** Elemento inicial, como ícone, foto de perfil/avatar ou imagem.
PDF
- **Trailing Content (trailingContent):** Elemento final à direita, como ícones, botões, meta-informações ou contadores.
PDF
- **Divider (HorizontalDivider):** Linha separadora horizontal.

Modos de Carregamento de Listas Dinâmicas

- **Carregamento Manual (Column + verticalScroll):** Renderiza todos os itens da lista de uma única vez na memória. É ineficiente para grandes volumes de dados.
PDF+ 1
- **Carregamento Otimizado (LazyColumn):** Renderiza dinamicamente apenas os itens visíveis na tela à medida que o usuário rola a lista. É o modo ideal e econômico para coleções dinâmicas de dados

Desenvolvimento Passo a Passo

Definir o Modelo de Dados: Criamos uma `data class` em Kotlin para representar as informações de cada usuário (nome, status online/offline e o recurso visual).

```
// Modelo para recursos locais
data class UserProfile(
    val name: String,
    val status: Boolean,
    val drawableId: Int
)

// Modelo alternativo para imagens via URL (remota)
// data class UserProfile(val name: String, val status: Boolean, val pictureUrl: String)
```

A classe armazena o nome (`name`), o status de conexão (`status` como `Boolean`) e a imagem associada.

Construir o Componente Visual da Foto: A foto de perfil exibe uma borda cuja cor muda dinamicamente conforme o status do usuário (verde para online, vermelho para offline)

```
// Modelo para recursos locais
data class UserProfile(
    val name: String,
    val status: Boolean,
    val drawableId: Int
)

// Modelo alternativo para imagens via URL (remota)
// data class UserProfile(val name: String, val status: Boolean, val pictureUrl: String)
```

O Card circular adiciona uma borda (`BorderStroke`) verde (`Color.Green`) caso `onlineStatus` seja verdadeiro, ou vermelha (`Color.Red`) se for falso.

Agrupar o Card Completo: Une a imagem e as informações de texto dentro de um layout de linha (`Row`) envolvido por um `Card`

```
@Composable
fun ProfileCard(userProfile: UserProfile) {
    Card(
        modifier = Modifier
            .padding(top = 8.dp, bottom = 4.dp, start = 16.dp, end = 16.dp)
            .fillMaxWidth()
            .wrapContentHeight(),
        elevation = CardDefaults.cardElevation(defaultElevation = 8.dp),
        colors = CardDefaults.cardColors(containerColor = Color.White)
    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.Start
        ) {
            ProfilePicture(userProfile.drawableId, userProfile.status)
            ProfileContent(userProfile.name, userProfile.status)
        }
    }
}
```

Organiza a foto à esquerda (leading) e os textos à direita dentro de um container com elevação.

Implementar a Lista Dinâmica Otimizada: Substituímos o loop manual for por um LazyColumn, garantindo alta performance de renderização.

```
@Composable
fun MainScreen() {
    Surface(
        modifier = Modifier
            .fillMaxSize()
            .safeDrawingPadding()
    ) {
        // Modo Otimizado: renderiza itens sob demanda na rolagem
        LazyColumn {
            items(userProfileList) { userProfile ->
                ProfileCard(userProfile = userProfile)
            }
        }
    }
}
```

A função items(userProfileList) instrui o LazyColumn a recompor apenas os elementos do conjunto userProfileList conforme aparecem na área visível da tela.